Newton School of Technology

6

UML Part 1

Class Diagrams

by Aditya Kumar

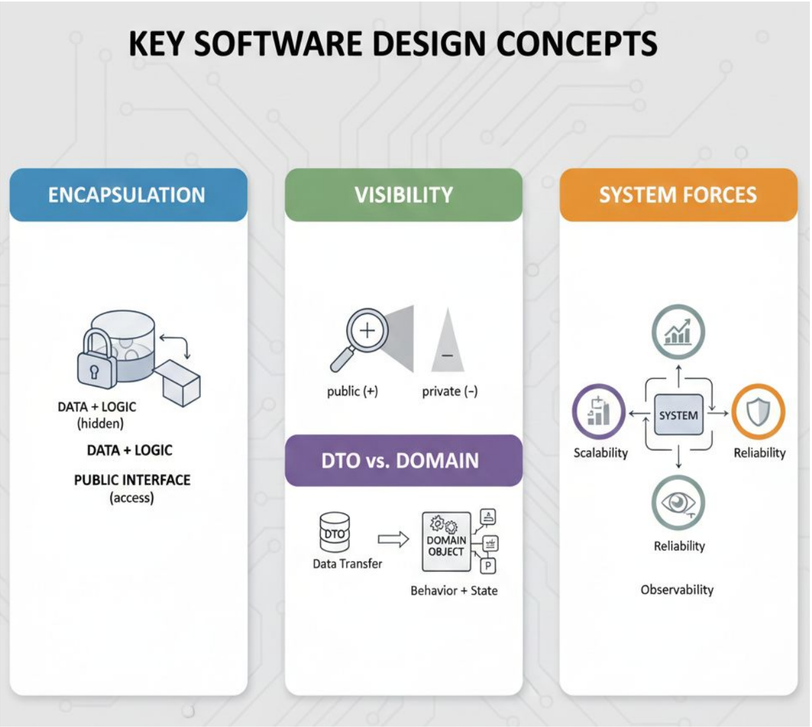
Join the lecture online on your dashboard.

Recap of Previous Lecture!

Quick Recap

Key Concepts We Covered:-

1. **Encapsulation** - Protecting system invariants
2. **Visibility Modifiers** - public vs private
3. **DTOs vs Domain Objects** - Data-only vs Rule-enforcing
4. **System Forces** - Scalability, Reliability, Observability



Quick Discussion:-

- What is an invariant you protected in last week's exercise?
- How does encapsulation help maintain data integrity?

UML

Imaging you are building your house, what is the first thing that you need before you start construction?

UML

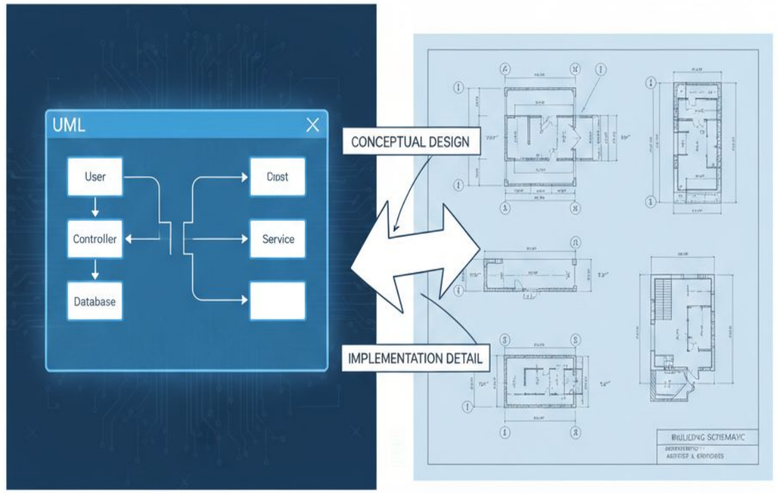
Imaging you are building your house, what is the first thing that you need before you start construction.

UML

Imagine Building your house without a plan, what can go wrong?

What is UML?

- UML is a **visual modeling language** for system design
- Provides a **standard way** to represent a system
- Similar to **engineering blueprints**
- Shows **structure and behavior** of a system
- Used by **developers, architects, and business teams**

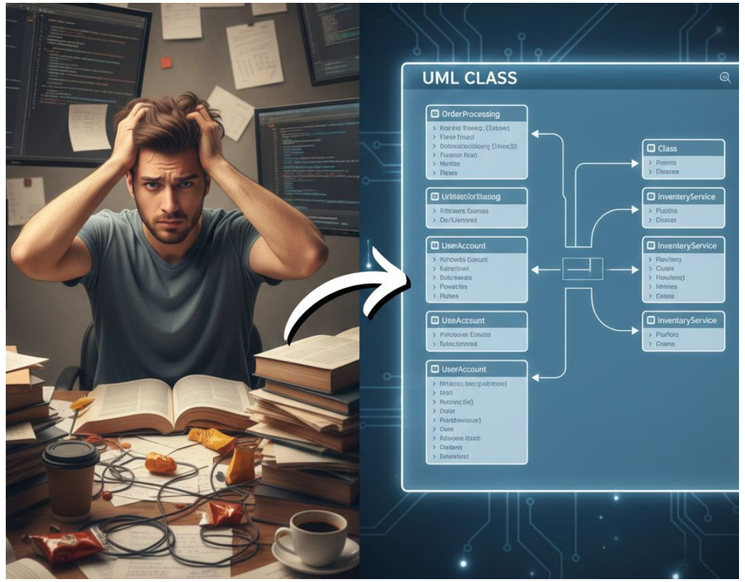


Need Of UML ?

Before UML (1997), every company used different notation. UML unified these into one standard.

Three Core Purposes:

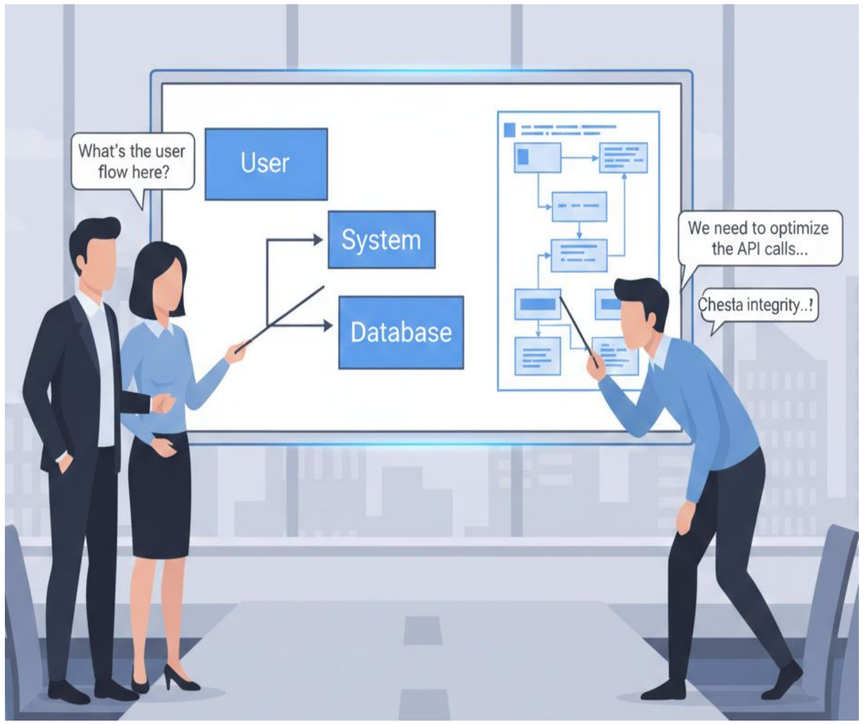
1. **Clarifies Thinking** - Forces us to formalize vague ideas
2. **Communicates Requirements** - Pictures speak louder than words
3. **Early Discovery** - Finds problems before coding



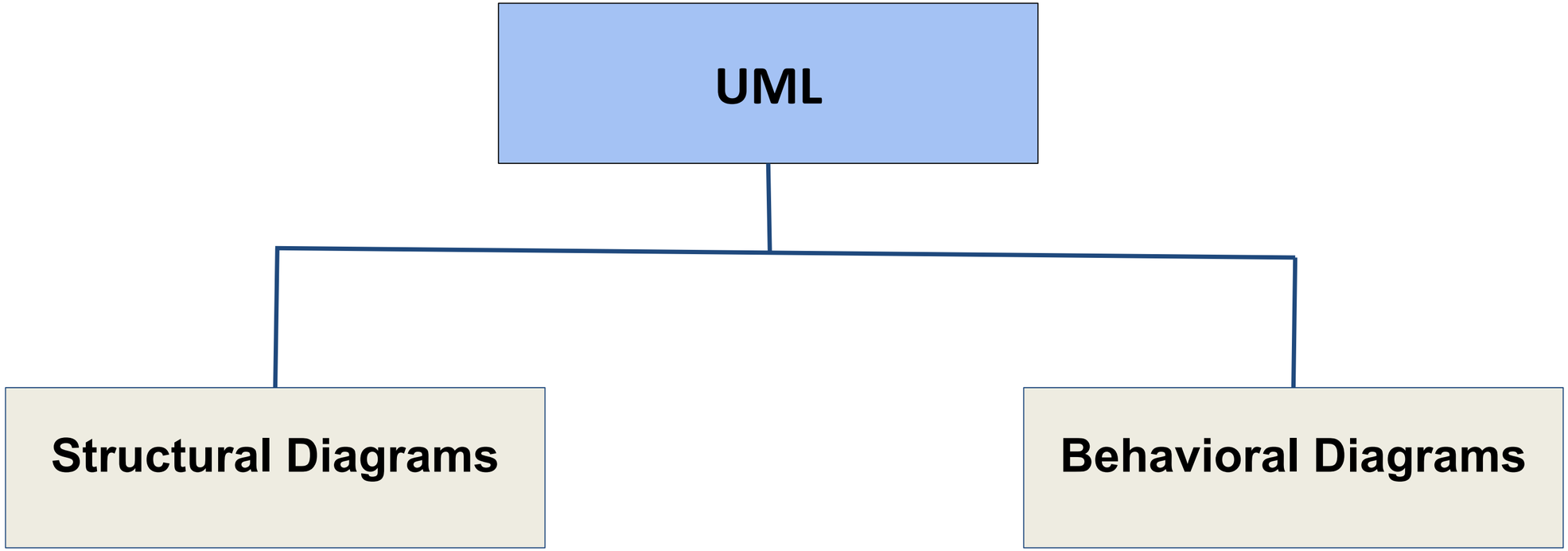
Building a house without blueprints = chaos. UML is the blueprint language for software.

Need Of UML?

- Enables clear communication across multiple teams
- Helps non-technical stakeholders understand the system
- Visualizes system structure, flow, and interactions
- Reduces misunderstandings and rework
- Saves time during development and maintenance



Types of UML Diagrams



Structural vs. Behavioral

Structural vs. Behavioral

- Wheels of a car
- The car moves
- Brakes of a bike
- The bike stops

Structural vs. Behavioral

Can a car move without wheels?

Structural vs. Behavioral

Both are needed! Structure without behavior = parts without purpose.

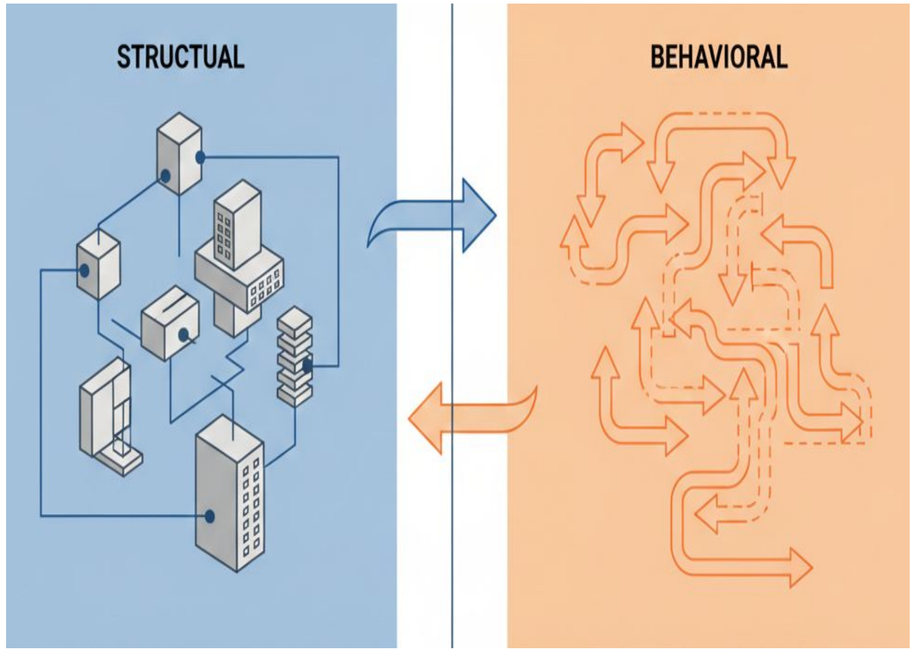
Structural vs. Behavioral

STRUCTURAL Diagrams (What the system IS)

- Focus: Static components and relationships
- Question: "What parts make up the system?"
- **Examples:** Class Diagram

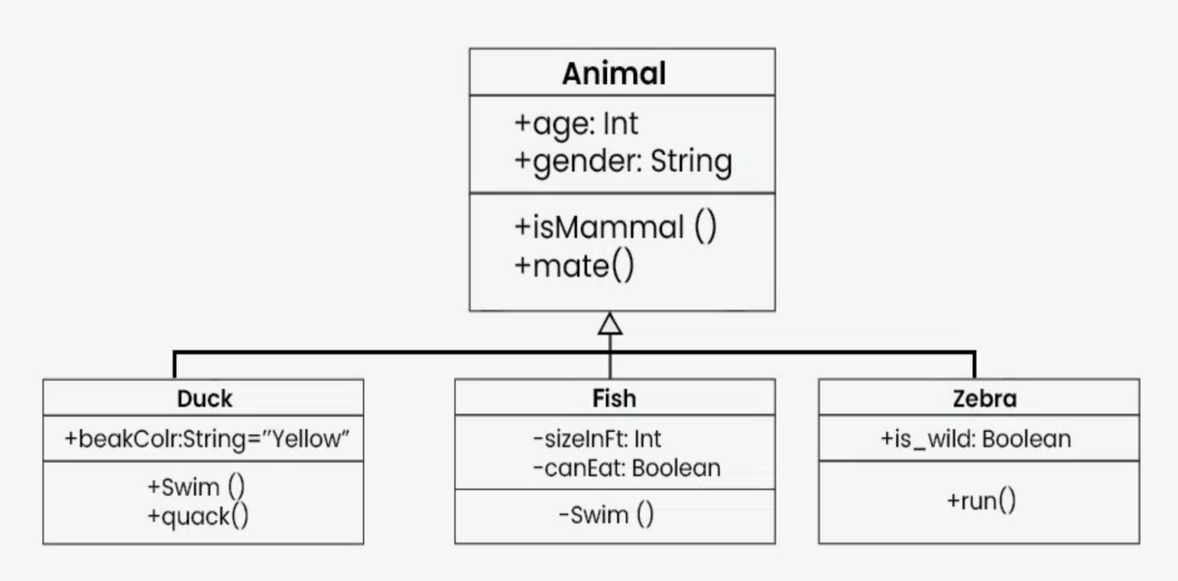
BEHAVIORAL Diagrams (How the system BEHAVES)

- Focus: Dynamic processes and interactions
- Question: "What does the system do over time?"
- **Examples:** Use Case Diagram ← TODAY, Sequence Diagram



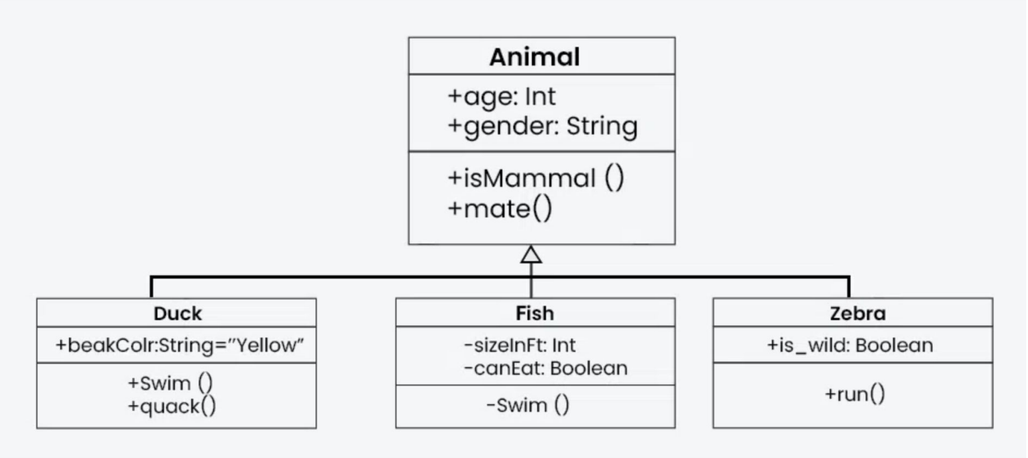
Class Diagram

A UML class diagram shows the structure of a system by displaying its classes, their attributes, methods, and the relationships between them.It helps the team understand how the system is organized and how components interact.



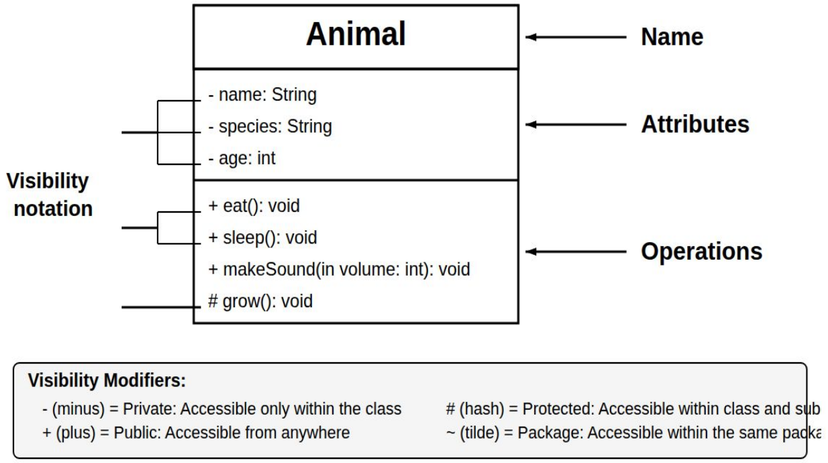
Class Diagram

- Represents classes, attributes, methods and relationships
- Helps communicate and document the software structure
- Makes it easier for developers and designers to understand the system



Class Notation

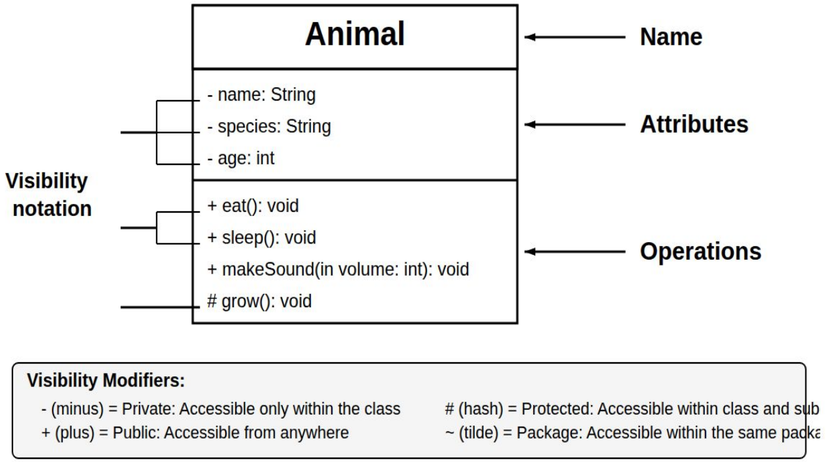
Classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods.



1. Class Name: The name of the class is typically written in the top compartment of the class box and is centered and bold.

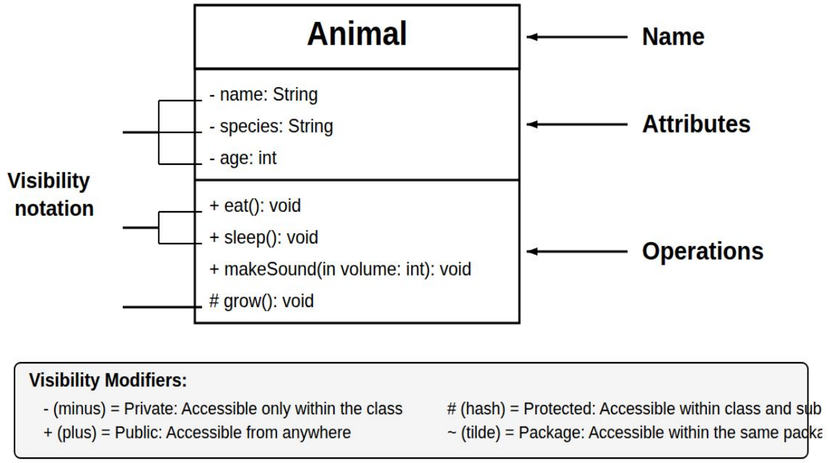
Class Notation

2. Attributes: Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.



Class Notation

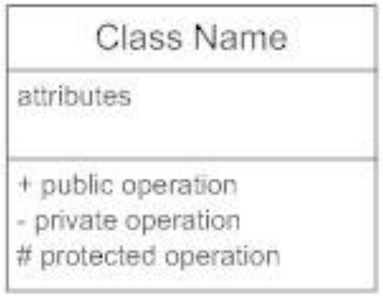
3. Methods: Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.



Class Notation

4. Visibility Notation: Visibility notations indicate the access level of attributes and methods. Common visibility notations include:

- + for public (visible to all classes)
- - for private (visible only within the class)
- # for protected (visible to subclasses)
- ~ for package or default visibility (visible to classes in the same package)



Visibility

Marker	Visibility
+	public
-	private
#	protected
~	package

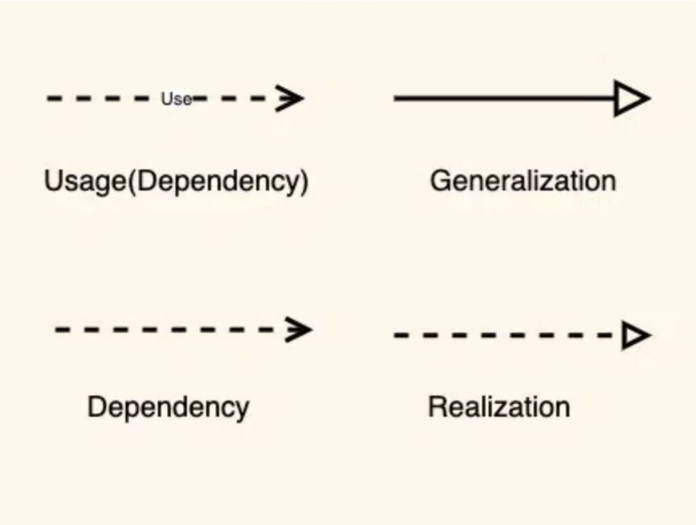
Relationships Between Classes

1. Generalization (Inheritance) - "Is-a" relationship; child inherits from parent (solid line with hollow arrow)

2. Realization - Class implements an interface (dashed line with hollow arrow)

3. Dependency - Weak reliance between classes; loosely coupled (dashed arrow)

4. Usage - Client uses supplier's services without owning it (dashed arrow)



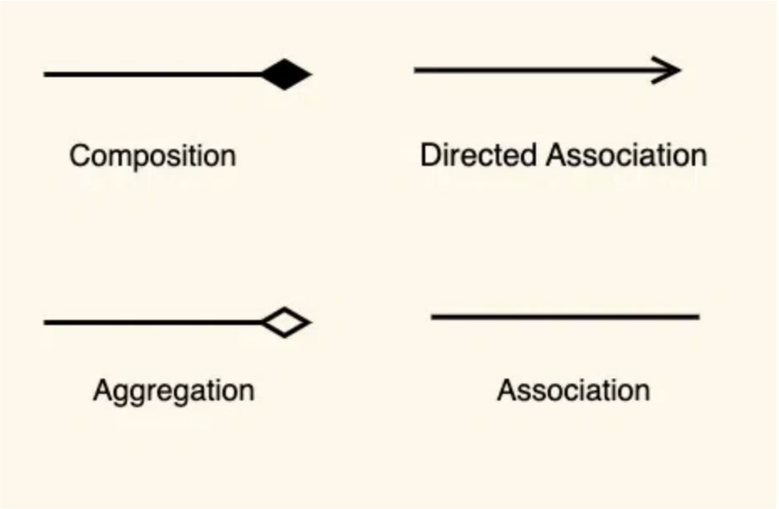
Relationships Between Classes

5. Association - Bi-directional connection between classes (solid line)

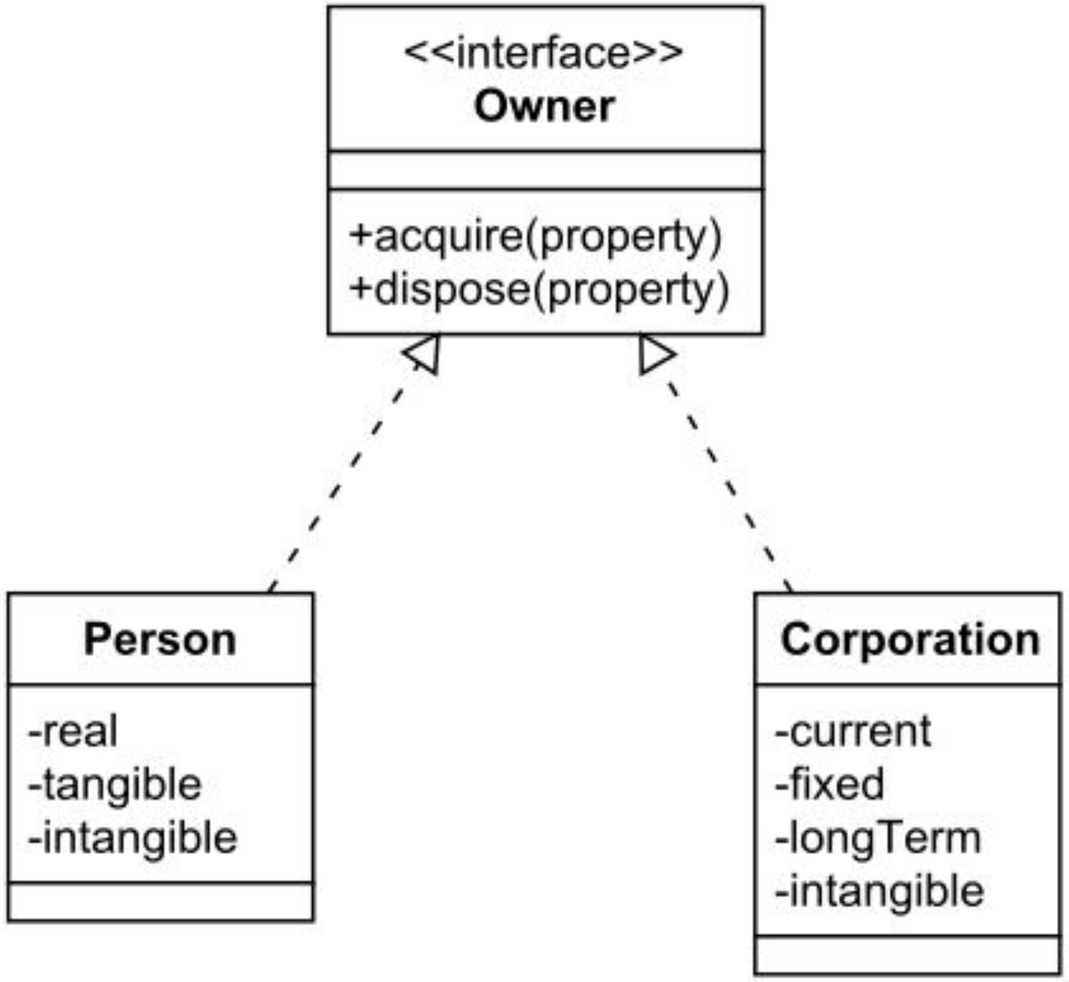
6. Directed Association - One-way association with specific direction (solid line with arrow)

7. Aggregation - "Whole-part" relationship; part can exist independently (hollow diamond)

8. Composition - Strong ownership; part cannot exist without whole (filled diamond)



Interface In UML



Purpose of Class Diagrams

- Best UML diagram to represent **OOP concepts**
- Enables **clear and efficient system design**
- Serves as a base for **component and deployment diagrams**
- Supports **forward and reverse engineering**

Benefits of Class Diagrams

Newton School of Technology

- Provide a clear view of **system architecture**
- Show **class relationships** (association, inheritance)
- Improve communication across **technical and non-technical teams**
- Guide developers during **implementation**
- Enable **code generation** in supported tools

Class Diagram Activity

Newton School of Technology

Design a Class Diagram for food ordering app like swiggy and zomato.

zomato SWIGGY

MCQ Time ..

Your Turn

Class Diagram Exercise

Newton School of Technology

Design a Class Diagram for a College System

System Requirements:

- A **Student** enrolls in multiple **Courses**
- Each **Course** has a name, code, and credits
- A **Teacher** teaches multiple **Courses**

Tasks:

Create a class diagram showing:

1. **Three classes:** Student, Course, Teacher
2. **Add 2-3 attributes** to each class
3. **Add 1-2 methods** to each class
4. **Show relationships** with lines

Summary & Takeaways

- **UML** is a vital visual communication tool for software systems.
- **Structural ≠ Behavioral** diagrams (what vs. how).
- **Structural Diagrams** show system structure and components (e.g., Class Diagram).
- **Behavioral Diagrams** show system behavior and interactions (e.g.Sequence Diagram).
- **Class Diagrams** define classes, attributes, methods, and relationships between objects.

Newton School of Technology

Thank You ..